



Ai2th

Posted on Mar 5 • Edited on Aug 5



1

Pockr | Part 4 — Docker Without Kernel Modules

[#docker](#) [#linux](#) [#android](#) [#networking](#)

Pockr — Running Docker on a Non-Rooted Android Phone (8 Part Series)

- 1 [Pockr — Running Docker on a Non-Rooted Android Phone](#)
- 2 [Pockr | Part 1 — The Idea and Architecture](#)
- ... **4 more parts...**
- 5 **Pockr | Part 4 — Docker Without Kernel Modules**
- 7 [Pockr | Part 6 — Test Results and What's Next](#)
- 8 [Pockr | Part 7 — Standalone App Deprecated & Merged into Linxr](#)

Docker Run Without iptables, bridge, or overlay2

Part 4 of 6 — building Pockr, a single APK that runs Docker on non-rooted Android.

← [Part 3: Bundling 50 Native Libraries](#)

Docker Starts — Then Immediately Fails

Once QEMU was running and Alpine booted, we launched Docker. It started, then failed silently. Containers wouldn't run. No useful error message.

The root cause: Alpine 3.19's Docker daemon defaults assume a full Linux kernel. Our QEMU kernel (`6.6.14-0-virt`) is stripped — it doesn't load `iptables`, `bridge`, `overlay2`, or `ip-masq` as kernel modules because there's no `/lib/modules` in the disk image.

What Was Missing

Feature Docker Expects	Kernel Module	Status in Our VM
Network filtering	<code>iptables</code> / <code>nf_tables</code>	Not available
Container networking	<code>bridge</code>	Not available
Efficient storage	<code>overlay2</code>	Not available
NAT/masquerade	<code>ip-masq</code>	Not available

Docker's default config tries to set up all of these on startup. Every one fails with `operation not supported`.

The Fix: Minimal Docker Config

We bake a custom `/etc/docker/daemon.json` into the Alpine base image:

```
{
  "iptables": false,
  "bridge": "none",
  "dns": [ "10.0.2.3", "8.8.8.8", "8.8.4.4" ],
  "ip-masq": false,
  "userland-proxy": false,
  "storage-driver": "vfs"
}
```

Setting	Why
<code>iptables: false</code>	Alpine's iptables uses nft backend — module unavailable
<code>bridge: none</code>	Bridge kernel module unavailable
<code>ip-masq: false</code>	No masquerading needed with host networking
<code>userland-proxy: false</code>	Avoids port proxy process startup failures
<code>storage-driver: vfs</code>	overlay2 needs kernel module; vfs always works

Container Networking: `--network host`

With no bridge, containers run with `--network host` — they share the VM's network stack directly.

The VM's network is QEMU SLIRP (user-mode networking). SLIRP gives the guest:

- An `eth0` at `10.0.2.15`
- Default gateway at `10.0.2.2`
- DNS proxy at `10.0.2.3`
- Port forwarding from Android host to guest

Containers inherit all of this. They can reach the internet, pull images from Docker Hub, and serve traffic on forwarded ports.

The DNS Trap

QEMU's built-in DNS proxy at `10.0.2.3` uses UDP. On real Android hardware (confirmed on Firebase Test Lab Pixel2.arm), UDP DNS is unreliable — Docker Hub pulls fail intermittently with:

```
Error response from daemon: Get "https://registry-1.docker.io/v2/":
dial tcp: lookup registry-1.docker.io: i/o timeout
```

Fix in `/etc/resolv.conf`:

```
nameserver 10.0.2.3
nameserver 8.8.8.8
nameserver 8.8.4.4
options timeout:2 attempts:2 use-vc
```

`use-vc` forces all DNS over **TCP**. With multiple fallback nameservers, Docker Hub pulls work reliably — confirmed with nginx (~78 seconds on Pixel2.arm, Android 11).

Storage: vfs vs overlay2

`vfs` is not space-efficient (it copies layers instead of linking them), but it works without kernel modules. For our use case — a VM with an 8 GB overlay disk — it's perfectly acceptable.

Pulled images are stored in the writable `user.qcow2` overlay and survive VM restarts. Once `nginx` is pulled, subsequent boots start it instantly with no re-download.

Next: [Part 5 — Debugging a VM Restart Loop](#)

GitHub: github.com/AI2TH/Pockr

Website: ai2th.github.io

Pockr Series — Docker in Your Pocket

Pockr = *Pocket* + *Docker*. A single Android APK that runs real Docker containers in your pocket — no root, no Termux, no PC required.

#	Post	Topic
	Intro	What is Pockr? Start here

#	Post	Topic
1	Part 1	The Idea and Architecture
2	Part 2	Executing Binaries — The SELinux Problem
3	Part 3	Bundling 50 Native Libraries
4	Part 4	Docker Without Kernel Modules
5	Part 5	Debugging the VM Restart Loop
6	Part 6	Test Results and What's Next

GitHub: github.com/AI2TH/Pockr

Website: ai2th.github.io

Pockr — Running Docker on a Non-Rooted Android Phone (8 Part Series)

1 Pockr — Running Docker on a Non-Rooted Android Phone

2 Pockr | Part 1 — The Idea and Architecture

... **4 more parts...**

5 **Pockr | Part 4 — Docker Without Kernel Modules**

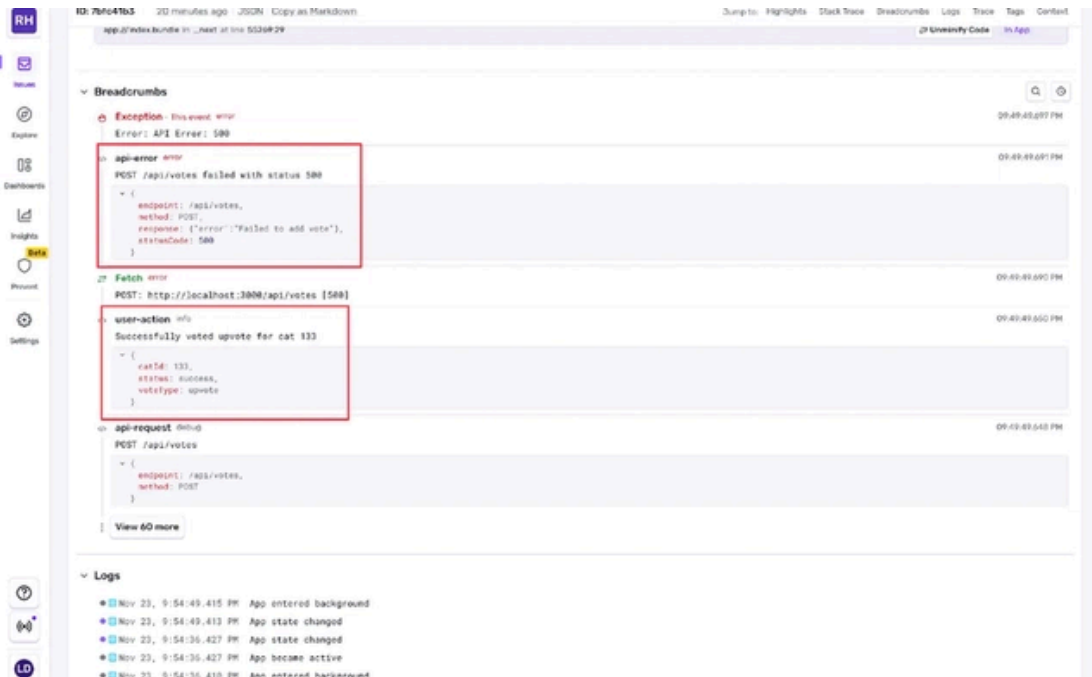
7 Pockr | Part 6 — Test Results and What's Next

8 Pockr | Part 7 — Standalone App Deprecated & Merged into Linxr



Sentry PROMOTED





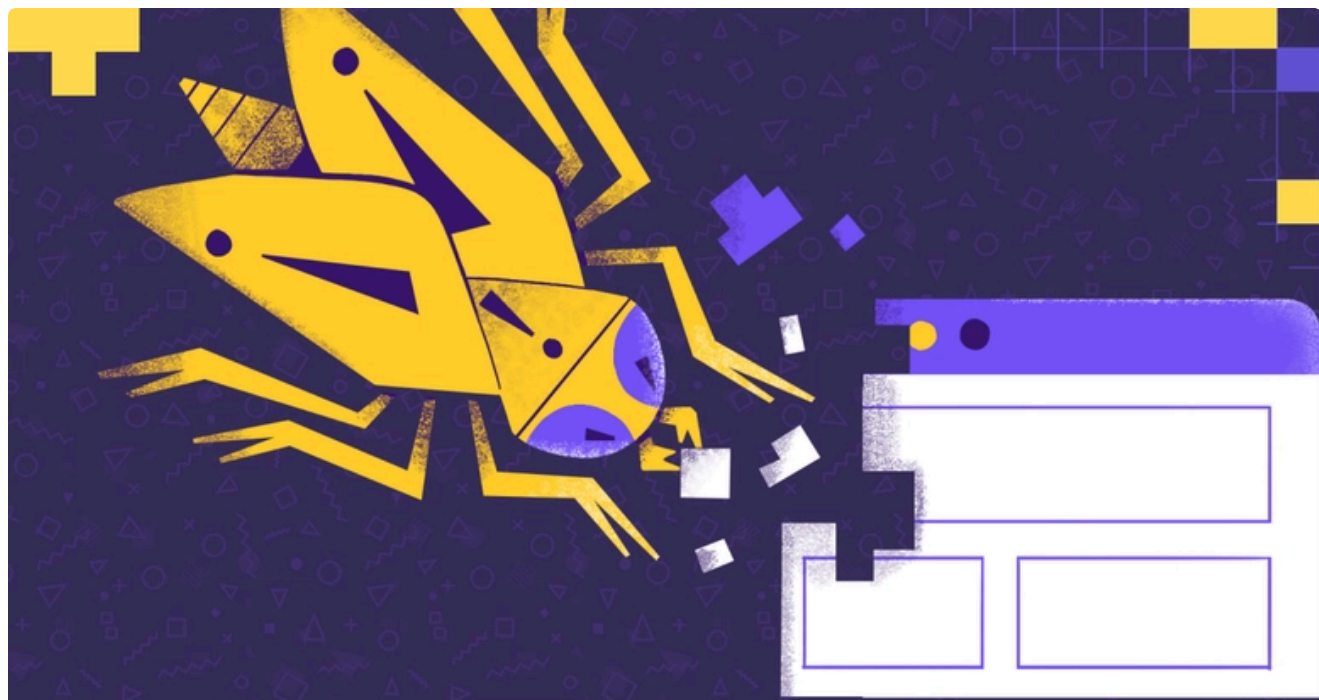
React Native logs that actually help you debug production issues

Improve your mobile debugging workflow with structured logs in React Native that provide actual context for production crashes.

[Read blog](#)

Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



Bugs are gross

In code and IRL. But we can only help with the code ones.

 **SENTRY**[**LEARN MORE**](#)

See why 4M developers consider Sentry, “not bad.”

Fixing code doesn't have to be the worst part of your day. Learn how Sentry can help.

[Learn more](#)

Ai2th

More from Ai2th

Linxr | Part 11 — The v3.0.0 Milestone & Official Release

#android #linux #docker #opensource

Linxr | Part 10 — Automated End-to-End Testing for Mobile Virtual Machines

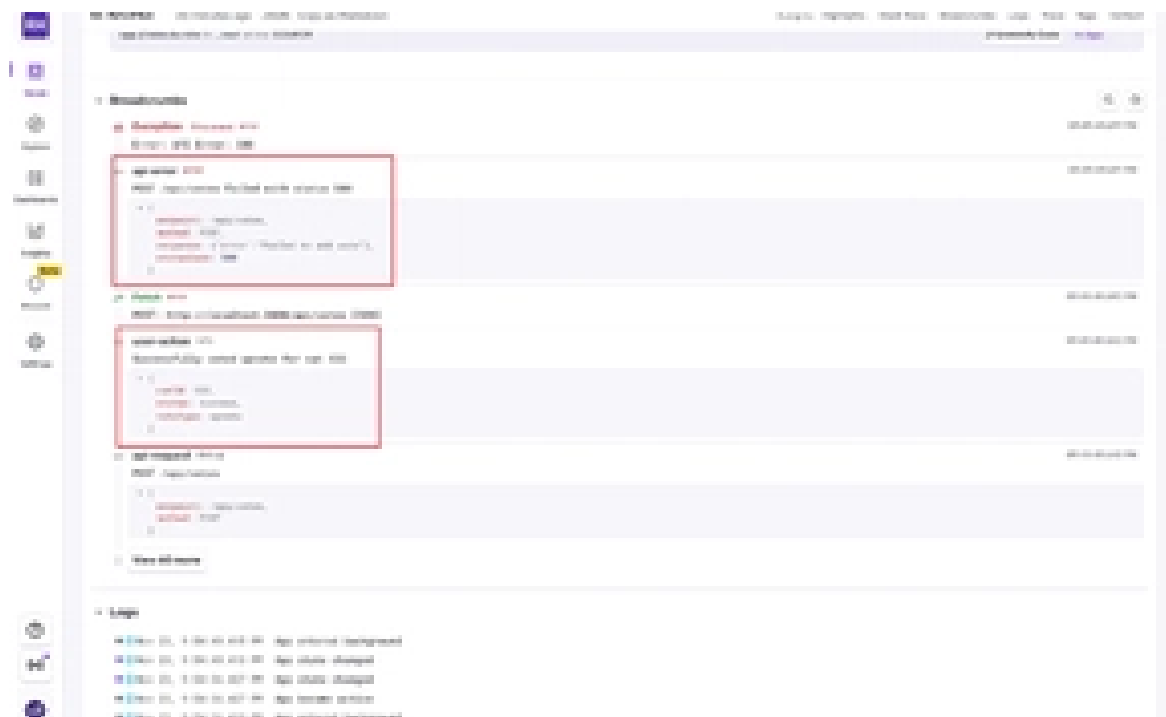
#testing #android #linux #ci

Linxr | Part 9 — Dynamic Storage Expansion & Automatic resize2fs on Boot

#android #linux #storage #sysadmin



Sentry PROMOTED



React Native logs that actually help you debug production issues

Improve your mobile debugging workflow with structured logs in React Native that provide actual context for production crashes.

[Read blog](#)
